



Programmation Fonctionnelle et Traduction des Langages

Rapport de Projet

Magron Mathis
Nancey Mathis

Département Sciences du Numérique
Deuxième année HPC & Big Data
2025 — 2026

Table des matières

1	Introduction	1
2	Extension du Langage RAT	1
3	Modifications des types (type.ml, type.mli)	2
4	Analyse Lexicale (lexer.mll) et Syntaxique (parser.mly)	3
4.1	Lexer	3
4.2	Parser	3
5	Arbres Abstraits (ast.ml)	3
6	Gestion des Identifiants (tds.ml, passeTdsRat.ml)	4
6.1	Table des Symboles (TDS)	4
7	Typage (passeTypRat.ml)	5
8	Placement Mémoire (passePlacementRat.ml)	6
9	Génération de Code TAM (passeCodeRatToTam.ml)	6
9.1	Génération de Code (TAM)	6
10	Jugements de typages	7
10.1	Les pointeurs	7
10.2	Les procédures	8
10.3	Le passage des paramètres par références	8
10.4	Les types énumérés	8
11	Conclusion	8

1 Introduction

Dans le cadre du cours de **Traduction des langages** nous avons introduit le langage **RAT** afin de comprendre comment fonctionne un compilateur en se basant sur la programmation fonctionnelle. Dans ce projet, nous étendons le compilateur du langage RAT, réalisé lors des travaux pratiques de traduction des langages, afin d'y ajouter de nouvelles fonctionnalités : les **pointeurs**, les **procédures**, le **passage de variables par référence** et les **types énumérés**.

Nous aborderons dans ce rapport pour chaque extension du langage : l'évolution de des AST, les jugements de typages, ainsi que des explications sur leur traitement. Une brève conclusion cloture ce rapport dans laquelle nous exposons les difficultés rencontrées ainsi que d'éventuelles améliorations.

2 Extension du Langage RAT

Le compilateur demandé doit être capable de traiter le langage **RAT** étendu avec :

1. des **pointeurs** ;
2. des **procédures** ;
3. des **paramètres passés par référence** ;
4. des **types énumérés**.

La grammaire vue en TP est modifiée comme indiqué ci-dessous :

1. $PROG' \rightarrow ENUM \star PROG \$$
2. $ENUM \rightarrow enum\ tid \{IDS\} ;$
3. $IDS \rightarrow tid \langle , tid \rangle \star$
4. $PROG \rightarrow FUN \star id\ BLOC$
5. $FUN \rightarrow TYPE\ id\ (DP)\ BLOC$
6. $BLOC \rightarrow \{ I \star \}$
7. $I \rightarrow TYPE\ id = E ;$
 $\quad | id = E ;$
8. $| A = E ;$
9. $| const\ id = entier ;$
10. $| print\ E ;$
11. $| if\ E\ BLOC\ else\ BLOC$
12. $| while\ E\ BLOC$
13. $| return\ E ;$
14. $| return ;$
15. $| id\ (CP) ;$
16. $DP \rightarrow \Lambda$
17. $| ref? TYPE\ id \langle , ref? TYPE\ id \rangle \star$
18. $TYPE \rightarrow bool$
19. $| int$
20. $| rat$
21. $| TYPE \star$
22. $| void$
23. $| tid$
5. $A \rightarrow id$
6. $| (\star A)$
7. $E \rightarrow id\ (CP)$
8. $| [E / E]$
9. $| num\ E$
10. $| denom\ E$
 $\quad | id$
11. $| A$
12. $| true$
13. $| false$
14. $| entier$
15. $| (E + E)$
16. $| (E \star E)$
17. $| (E = E)$
18. $| (E < E)$
19. $| (E)$
20. $| null$
21. $| (new\ TYPE)$
22. $| \& id$
23. $| ref\ id$
24. $| tid$
25. $CP \rightarrow \Lambda$
26. $| E \langle , E \rangle \star$

Pour simplifier la lecture, les guillemets ' ' sont omis autour des caractères. Les symboles issus de l'EBNF sont notés en gris :

- la répétition est notée $\star$, à distinguer du caractère '*' ;
- l'optionnalité est notée $?$;
- le parenthésage est réalisé avec $\langle \text{et} \rangle$.

3 Modifications des types (type.ml, type.mli)

Le type `typ` a été étendu avec trois nouveaux constructeurs pour supporter les nouvelles fonctionnalités :

Listing 1 – Modification de `Ast.typ`

```

1 type typ =
2   | Bool
3   | Int
4   | Rat
5   | Ptr of typ      (* Extension Pointeurs *)
6   | Enum of string  (* Extension Enumeres *)
7   | Void            (* Extension Procedures *)

```

Les fonctions auxiliaires ont été mises à jour : - `string_of_type` : affichage des nouveaux types (ex : “Int*”, “Void”, “Enum Mois”). - `getTaille` : - Pointeur `_` $\rightarrow$ taille 1 (une adresse). - Enum `_` $\rightarrow$ taille 1 (représenté par un entier). - Void $\rightarrow$ taille 0. - `est_compatible` : - Pointeur `t1` compatible avec Pointeur `t2` si `t1` compatible avec `t2`. - Void compatible avec Void. - Enum `n1` compatible avec Enum `n2` si `n1 = n2` (égalité structurelle des noms).

Une variable est physiquement identifiée de façon unique par son adresse, c’est-à-dire l’adresse de l’emplacement mémoire qui contient sa valeur. Un pointeur est une variable qui contient l’adresse d’un autre objet informatique (une “variable de variable” en somme).

4 Analyse Lexicale (lexer.mll) et Syntaxique (parser.mly)

4.1 Lexer

Ajout de nouveaux tokens pour les mots-clés et symboles : - **Pointeurs** : NULL (“null”), NEW (“new”), ADDR (“&”) - **Procédures** : VOID (“void”), RETURN (“return” généralisé) - **Références** : REF (“ref”) - **Enoms** : ENUM (“enum”), TID (identifiant de type commençant par une majuscule)

4.2 Parser

La grammaire a été modifiée pour intégrer les affectables et les nouvelles constructions :

Affectables Introduction de la règle `a` pour distinguer ce qui peut être à gauche d’une affectation (l-value) :

$$a ::= ID \mid (* a) \quad (\text{Variable ou dérérérencement})$$

L’instruction d’affectation devient ainsi `a = e`.

Pointeurs Ajout des constructions suivantes :

- **Expressions** : `null`, `(new type)`, `&id`.
- **Types** : `type *`.

Procédures Distinction des fonctions sans retour :

- **Instructions** : `id(args)`; (Appel), `return`; (Retour Void).
- **Type** : `void`.

Références Gestion du passage par référence :

- **Paramètres** : `ref type id` dans la définition de fonction.
- **Appel** : `ref id` lors de l’appel pour passer l’adresse.

Enums Définition et utilisation des types énumérés :

- **Déclaration globale** : `enum Tid {Val1, Val2};`.
- **Utilisation** : Les valeurs sont identifiées via leur type (ex : `Val1`).

5 Arbres Abstraits (ast.ml)

Les modules de définition de l’Arbre Abstrait (`AstSyntax`, `AstTds`, `AstType`, `AstPlacement`) ont été enrichis de manière similaire pour supporter ces nouvelles structures.

Affectables

Un nouveau type `affectable` a été introduit pour distinguer les expressions pouvant apparaître à gauche d’une affectation (l-values), c’est-à-dire les variables et les pointeurs dérérérencés.

Listing 2 – Définition du type affectable

```
1 type affectable =  
2   | Ident of info  
3   | Deref of affectable
```

Cette modification entraîne deux changements majeurs dans l’AST :

- L’instruction `Affectation` prend désormais un `affectable` en premier argument (au lieu d’un `string` ou d’une `info`).
- L’expression `Ident` disparaît et est remplacée par un constructeur plus générique : `Acces of affectable`.

Expressions

De nouveaux constructeurs ont été ajoutés au type `expression` pour gérer la manipulation mémoire et les énumérations :

- `Null` : Représente la constante de pointeur nul.
- `New of typ` : Correspond à l’allocation dynamique de mémoire.
- `Adresse of info` : Représente la prise d’adresse d’une variable (`&x`).
- `ValeurEnum of info` : Permet l’utilisation d’une valeur issue d’un type énuméré.

Instructions

Les instructions ont été étendues pour supporter les appels de procédures et la sémantique du retour vide :

- `RetourVoid` : Ajouté pour gérer l’instruction `return` ; spécifique aux procédures.
- `AppelProcedure` : Séparé de `AppelFonction`, ce constructeur gère les appels qui ne produisent pas de valeur de retour.
- **Gestion des références** : Les constructeurs `AppelFonction` et `AppelProcedure` incluent désormais une liste d’expressions associée à un booléen `is_ref` pour chaque paramètre, indiquant si le passage se fait par valeur ou par référence.

6 Gestion des Identifiants (tds.ml, passeTdsRat.ml)

6.1 Table des Symboles (TDS)

Le module de gestion de la table des symboles (`tds.ml`) et la passe associée (`passeTdsRat.ml`) ont été adaptés pour stocker et vérifier les nouvelles propriétés sémantiques nécessaires aux extensions.

Modification des Structures (tds.ml)

Le type `info_ast`, qui stocke les métadonnées associées aux identifiants, a été étendu pour supporter le passage par référence et les énumérations :

- **Fonctions** : Le constructeur `InfoFun` s’enrichit d’une liste de booléens (`bool list`). Chaque booléen correspond à un paramètre et indique s’il est passé par référence (`true`) ou par valeur (`false`).
- **Énumérations** : Un nouveau constructeur `InfoEnum` permet de stocker les constructeurs de types énumérés, en associant le nom de la valeur à son type parent.

Listing 3 – Extension du type info dans tds.ml

```

1 type info_ast =
2   | InfoVar of string * typ * int * string
3   | InfoConst of string * int
4   | InfoFun of string * typ * typ list * bool list
5   | InfoEnum of string * string

```

Passe d'Analyse des Identifiants (passeTdsRat.ml)

La logique de la passe TDS a été revue pour intégrer les règles suivantes :

- **Affectables** : L'analyse des affectables est devenue récursive pour traiter les déréférencements (pointeurs). Le compilateur vérifie désormais strictement la nature de l'identifiant à gauche d'une affectation (interdiction d'affecter une constante).
- **Références** : Lors de la définition d'une fonction, l'information `is_ref` des paramètres est capturée et propagée dans la TDS. Cela permettra à la passe de typage de valider la cohérence des appels.
- **Énumérations** : Les valeurs des énumérations sont enregistrées dans la TDS globale (Main) avant le traitement du corps du programme. Cela garantit qu'elles sont uniques et accessibles partout.
- **Exceptions** : Une exception `MauvaiseUtilisationIdentifiant` est systématiquement levée si l'utilisateur tente d'affecter une valeur à une constante ou d'utiliser une fonction comme une variable standard.

7 Typage (passeTypRat.ml)

Cette passe a pour rôle de vérifier la cohérence des types dans l'arbre abstrait décoré et de valider les contraintes spécifiques introduites par les extensions.

Traitement des Pointeurs

La gestion des pointeurs introduit de nouvelles règles de résolution de types :

- `New t` : L'expression est typée en `Pointeur t`.
- `Adresse (InfoVar t)` : La prise d'adresse d'une variable de type `t` renvoie un `Pointeur t`.
- `Deref e` : Si l'expression `e` est de type `Pointeur t`, le déréférencement est de type `t`. Une erreur est levée si `e` n'est pas un pointeur.
- `Null` : Cette constante est typée avec un type polymorphe `Pointeur Undefined`, la rendant compatible avec n'importe quel type de pointeur.

Gestion des Procédures

La distinction entre fonctions et procédures impose des vérifications strictes sur les instructions de retour :

- L'instruction `return`; (sans expression) est validée uniquement si la fonction courante a été déclarée avec le type de retour `void`.
- Inversement, l'instruction `return expr`; est interdite dans une procédure.
- Lors d'un appel, le compilateur vérifie que l'identifiant appelé correspond bien à une procédure (type de retour `void`) et non à une fonction retournant une valeur.

Validation des Références

Le passage par référence nécessite une cohérence parfaite entre la signature de la fonction et l'appel :

- Si un paramètre est déclaré avec `ref`, l'appelant **doit** impérativement fournir une référence explicite (`ref variable`).
- Il est interdit de passer une constante ou une expression temporaire (r-value) par référence.
- En cas de disparité (ex : paramètre `ref` mais appel par valeur, ou inversement), l'exception `RefIncoherent` est levée.

Types Énumérés

Outre la résolution classique des identifiants d'énumération, la passe de typage surcharge l'opérateur d'égalité :

- L'opération `e1 = e2` est autorisée entre deux énumérations si et seulement si elles appartiennent au **même** type énuméré.
- Dans le cas contraire, une exception `TypeBinaireInattendu` est levée.

8 Placement Mémoire (`passPlacementRat.ml`)

L'adaptation du calcul des tailles et des adresses mémoire (`passPlacementRat.ml`) a été réalisée pour prendre en compte les spécificités des nouvelles constructions.

- **Pointeurs** : La taille associée à un type pointeur est toujours de **1** mot mémoire, car il ne stocke qu'une adresse, quelle que soit la complexité du type pointé.
- **Références** : Le passage par référence modifie le calcul de la taille des paramètres :
 - Un paramètre passé par référence a une taille de **1** (on manipule l'adresse de la variable originale, pas une copie de sa valeur).
 - Dans la pile d'exécution, ce paramètre occupe donc systématiquement un seul emplacement (1 mot), même si la structure référencée est complexe.
- **Énumérations** : Les variables de type énuméré sont représentées en interne par des entiers. Leur taille est donc fixée à **1**.
- **Procédures** : La gestion du retour est adaptée pour le type `void` :
 - L'instruction `RetourVoid` (ou la fin implicite de la procédure) calcule la taille des paramètres pour nettoyer la pile.
 - Contrairement au retour de fonction classique, aucune valeur de retour n'est empilée à l'emplacement du résultat.

9 Génération de Code TAM (`passCodeRatToTam.ml`)

Voici la dernière partie du code LaTeX concernant la Génération de Code.

J'ai utilisé des listes de description (`description`) pour distinguer clairement les modes lecture/écriture et mis en forme les instructions TAM avec pour qu'elles ressortent bien visuellement. Extrait de code

9.1 Génération de Code (TAM)

L'étape de génération de code s'appuie sur le module `Tam.ml` pour traduire les nouvelles constructions en instructions de la Machine Abstraite.

Gestion des Pointeurs et Affectables

L'accès à un *affectable* est traité différemment selon le contexte : en lecture (expression) ou en écriture (affectation).

Accès à une Variable (x)

- **Lecture** : `LOAD (taille) dep[reg]`
- **Écriture** : `STORE (taille) dep[reg]`

Déréférencement ($*p$)

Le code généré évalue d'abord l'expression p , laissant une adresse au sommet de la pile.

- **Lecture** : `LOADI (taille)` (Charge la valeur située à l'adresse indiquée au sommet).
- **Écriture** : `STOREI (taille)` (Stocke la valeur du sommet à l'adresse située juste en dessous).

Opérations Spécifiques aux Pointeurs

- **Allocation** (`new type`) : On empile la taille requise (`LOADL taille`) puis on appelle `SUBR MAlloc`. L'adresse allouée dans le tas est retournée au sommet de la pile.
- **Prise d'adresse** (`&variable`) : L'instruction `LOADA dep[reg]` est utilisée pour empiler l'adresse de la variable (et non sa valeur).
- **Null** : Représenté simplement par l'entier 0 (`LOADL 0`).

Implémentation des Références

Le passage par référence est implémenté en traitant les paramètres comme des pointeurs implicites.

- **Lors de l'appel** : Si le paramètre est déclaré `ref`, on passe l'adresse mémoire :
 - Pour une variable : `LOADA dep[reg]`.
 - Pour un déréférencement existant : on évalue simplement le pointeur.
- **Dans la fonction appelée** : Le paramètre local contient une adresse. Pour accéder à la "vraie" valeur :
 1. `LOAD` : Récupération de l'adresse stockée dans le paramètre.
 2. `LOADI` ou `STOREI` : Manipulation de la donnée distante.

Note d'implémentation : Cette logique est unifiée avec celle des pointeurs et des déréférencements explicites via l'abstraction des affectables.

Types Énumérés

Les énumérations sont gérées comme des entiers pour la machine cible :

- **Valeurs** : Chaque valeur d'enum est convertie en un entier unique (via un hachage du nom) généré par `LOADL hash_val`.
- **Égalité** : La comparaison s'effectue avec l'instruction standard `SUBR IEq`.

10 Jugements de typages

10.1 Les pointeurs

Nous définissons ici les règles d'inférences pour les pointeurs du langage. On note σ l'environnement de typage.

$\frac{\text{NULL}}{\sigma \vdash \text{null} : * \tau}$	$\frac{\text{NEW}}{\sigma \vdash (\text{new } \tau) : * \tau}$	$\frac{\text{ADDR} \quad \sigma(\text{id}) = \tau}{\sigma \vdash \&\text{id} : * \tau}$	$\frac{\text{DEREF} \quad \sigma \vdash E : * \tau}{\sigma \vdash (*E) : \tau}$
--	--	---	---

10.2 Les procédures

$$\begin{array}{c}
 \text{CALL-PROC} \\
 \frac{\sigma(f) = \tau_1 \times \dots \times \tau_n \rightarrow \text{void} \quad \forall i, \sigma \vdash E_i : \tau_i}{\sigma \vdash f(E_1, \dots, E_n) : \text{void}}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{RET-VOID} \\
 \frac{\text{Contexte actuel} = \text{void}}{\sigma \vdash \text{return}; : \text{void}}
 \end{array}$$

10.3 Le passage des paramètres par références

Lors de l'appel d'une fonction, on distingue le passage par valeur (expression classique) du passage par référence (variable explicite).

Soit un paramètre formel p_i de type attendu $T_{attendu}$.

$$\begin{array}{c}
 \text{ARG-VAL} \\
 \frac{p_i : \tau \quad \sigma \vdash E : \tau}{\text{Arg } E \text{ valide pour } p_i}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{ARG-REF} \\
 \frac{p_i : \text{ref } \tau \quad \sigma(\text{id}) = \tau}{\text{Arg (ref id) valide pour } p_i}
 \end{array}$$

10.4 Les types énumérés

L'égalité est surchargée pour permettre la comparaison de deux identifiants du même type énuméré.

$$\begin{array}{c}
 \text{ENUM-VAL} \\
 \frac{\text{id} \in \text{Constructeurs}(T_{enum})}{\sigma \vdash \text{id} : T_{enum}}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{ENUM-EQ} \\
 \frac{\sigma \vdash E_1 : T \quad \sigma \vdash E_2 : T \quad T \text{ est un enum}}{\sigma \vdash (E_1 = E_2) : \text{bool}}
 \end{array}$$

11 Conclusion

Ce projet nous a permis d'étendre le compilateur du langage RAT pour y intégrer la gestion des pointeurs, des procédures, du passage par référence et des types énumérés. Toutes les fonctionnalités demandées ont été implémentés et validées par des tests, couvrant ainsi toute la chaîne : de l'analyse syntaxique jusqu'à la génération de code TAM. La difficulté principale a été l'implémentation des passes en programmation fonctionnelle (en OCaml) pendant les TP.